Chapter 3: Frontend Authentication

Welcome back! In our previous chapter, we explored the powerful "Interview AI Service" on the backend, which processes your resume and job descriptions to generate smart reports. That's a lot of valuable, personal information! But how do we make sure that only *you* can access *your* private interview reports and use the AI features connected to *your* account?

This is where "Frontend Authentication" comes into play.

What Problem Are We Solving?

Imagine you're using our interview-ai-yt application. You've created some fantastic interview reports, and you don't want just anyone to see them. You also want to easily log in, and have the app remember you next time you visit.

The "Frontend Authentication" is like the **security desk** of our application. It's the part that:

1. Provides login and registration forms for you to create and access your account.
2. Remembers if you're currently logged in, so you don't have to enter your password every time you visit a new page.
3. Helps different parts of the app (like your "My Reports" page) understand if they should show you your private data or prompt you to log in.
4. Handles sending your login/registration details securely to the backend (the "main security office") and processes its response.

Essentially, it gives you all the tools on the screen to manage your account and ensures the app knows *who you are* at all times.

Key Concepts: Your Security Tools

Just like with our interview management, the frontend uses "hooks" and "services" to handle authentication smoothly.

1. **"Hooks" (like useAuth):** This is your main toolbox for anything related to user accounts. It's a special function that gives your app's different screens (components) all the actions they need – like logging in, registering, logging out, and knowing if a user is currently logged in. Think of

it as the friendly security guard who gives you the sign-in sheet and checks your ID.
2. **"Services" (like auth.api.js):** These are like dedicated messengers for authentication. When useAuth needs to send your login details or check your status with the backend, it tells a "service" to go handle that communication. This keeps the UI code clean and focused on showing forms and messages. This messenger knows *exactly* how to talk to the backend's security system.
3. **User State:** This is a fancy term for how the frontend remembers *who* is currently logged in. If you're logged in, the useAuth hook will have information about you (like your username and email). If you're not logged in, it will know that too.
4. **Loading State:** Just like generating an AI report, logging in or registering takes a moment. The loading state tells the app if an authentication action is currently happening (e.g., waiting for the backend to verify your password), so it can show a "Loading..." message or disable buttons.

How to Use Frontend Authentication

As a user, you'll interact with login and registration forms. As a developer, the main tool you'll use is the useAuth hook.

Use Case 1: Registering a New Account

Let's say a new user wants to sign up for our interview-ai-yt app. They'll fill out a form with a username, email, and password.

Here's a simplified example of how a frontend component might use our useAuth hook to do this:

```
// A simplified React component for registration
import React, { useState } from 'react';
import { useAuth } from '../../features/auth/hooks/useAuth';

function RegisterForm() {
  const { handleRegister, loading, user } = useAuth(); // Get the register function, loading state,
and user info
  const [username, setUsername] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const handleSubmit = async () => {
    await handleRegister({ username, email, password });
    // After successful registration, 'user' will be updated and you might navigate to a dashboard
```

```
  if (user) {
    console.log("Welcome, new user:", user.username);
  }
 };

 if (loading) return <p>Registering your account...</p>;

 return (
  <div>
   {/* ... input fields for username, email, password ... */}
   <button onClick={handleSubmit}>Sign Up</button>
  </div>
 );
}
```

Explanation:
The RegisterForm component uses the useAuth hook to get the handleRegister function. When the user submits the form, handleRegister is called with their details. The loading state lets us show a message, and if successful, the user object will be updated, indicating they are now logged in.

Use Case 2: Logging In to an Existing Account

A returning user wants to access their account.

```
// A simplified React component for login
import React, { useState } from 'react';
import { useAuth } from '../../features/auth/hooks/useAuth';

function LoginForm() {
 const { handleLogin, loading, user } = useAuth(); // Get login function, loading, and user
 const [email, setEmail] = useState('');
 const [password, setPassword] = useState('');

 const handleSubmit = async () => {
  await handleLogin({ email, password });
  if (user) {
    console.log("Welcome back, user:", user.email);
  }
 };

 if (loading) return <p>Logging in...</p>;
```

```
  return (
    <div>
      {/* ... input fields for email, password ... */}
      <button onClick={handleSubmit}>Login</button>
    </div>
  );
}
```

Explanation:
Similar to registration, LoginForm uses handleLogin from useAuth. It sends the email and password, and user will be updated if the login is successful.

Use Case 3: Staying Logged In (Checking Status)

What happens when you close the browser and reopen the app? You don't want to log in every time! useAuth handles this automatically.

When your application first loads, the useAuth hook intelligently checks if you're already logged in from a previous session. This happens behind the scenes, using a special request to the backend.

```
// This happens automatically inside our useAuth hook (no component code needed for this)
// ...
export const useAuth = () => {
  const context = useContext(AuthContext);
  const { setUser, setLoading } = context;

  // This part runs once when the app starts
  useEffect(() => {
    const getAndSetUser = async () => {
      try {
        const data = await getMe(); // Ask the backend: "Am I logged in?"
        setUser(data.user); // If yes, save the user data
      } catch (err) {
        // Not logged in or session expired
      } finally {
        setLoading(false); // Done checking
      }
    };
    getAndSetUser(); // Call the check function
  }, []); // Empty array means run once on component mount
```

```
  // ... handleLogin, handleRegister, handleLogout functions ...
  return { user: context.user, loading: context.loading /* ... */ };
};
```

Explanation:
The useEffect block inside useAuth runs once when the app first loads. It calls getMe() (a function from our auth.api.js service) which asks the backend, "Do you recognize me from a previous login?". If the backend says yes, it provides the user's information, and the useAuth hook updates the user state, making the user appear logged in without needing to type their credentials again.

Use Case 4: Logging Out

When you're done and want to secure your session.

```
// A simplified React component for logging out
import React from 'react';
import { useAuth } from '../../features/auth/hooks/useAuth';

function Dashboard() {
  const { handleLogout, loading, user } = useAuth(); // Get logout function, loading, and user

  const handleSignOut = async () => {
    await handleLogout();
    console.log("Logged out successfully.");
  };

  if (loading) return <p>Logging out...</p>;
  if (!user) return <p>You are logged out.</p>; // Show if no user is logged in

  return (
    <div>
      <h2>Hello, {user.username}!</h2>
      {/* ... your private content ... */}
      <button onClick={handleSignOut}>Logout</button>
    </div>
  );
}
```

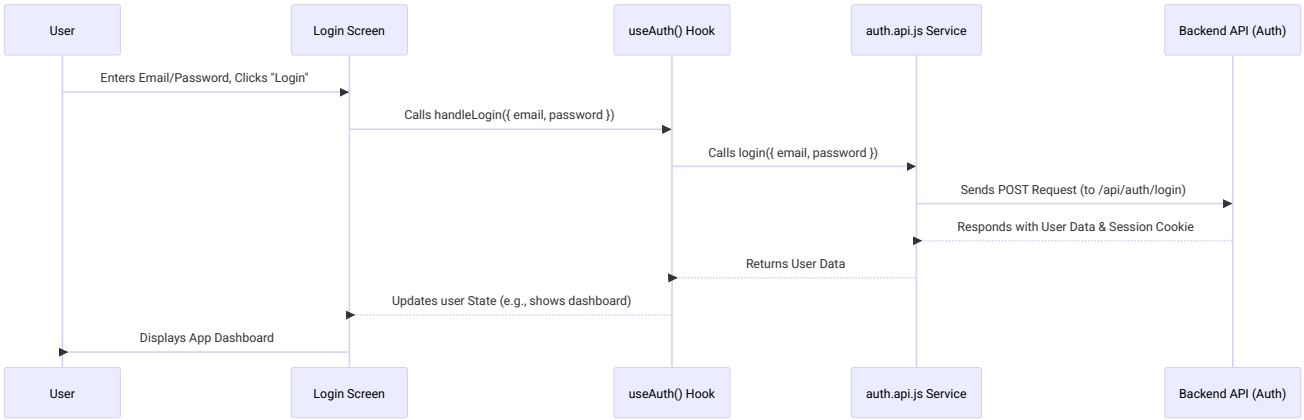
Explanation:
The Dashboard component gets handleLogout from useAuth. When the button is clicked, handleLogout is called. This tells the backend to end the session, and useAuth then clears the user state, reflecting that no one is currently logged in.

Under the Hood: How It All Works

Let's peek behind the curtain to understand how these authentication actions happen. When you call handleLogin() from useAuth, a few steps occur:

1. **Your Component Asks the Hook:** Your React component (the screen you see) asks the useAuth hook to perform an action (like logging in).
2. **The Hook Asks a Service:** The useAuth hook then tells the appropriate "service" (from auth.api.js) what needs to be done.
3. **The Service Talks to the Backend:** The service acts as a messenger, sending a request (with your email/password) to our backend (the "security office" where the actual user accounts are stored). This backend is powered by our User Authentication System.
4. **Backend Does the Work:** The backend receives the request, verifies your credentials, and if everything is correct, it tells the service that you are logged in (and might send back some basic user information and a special "cookie" to remember you).
5. **Service Brings Response Back to the Hook:** The service receives the response from the backend.
6. **Hook Updates Your Component:** The useAuth hook then receives this response and updates the user and loading data it provides to your component, which in turn updates what you see on the screen.

Here's a simple diagram illustrating this flow for a login:



Diving Deeper into the Code

Let's look at the actual code for useAuth and auth.api.js to see how these steps translate to our project.

1. **The useAuth Hook (Frontend/src/features/auth/hooks/useAuth.js)**

This file contains the useAuth hook. It's responsible for providing easy-to-use functions and managing loading states and user data globally within the frontend.

```
// Frontend/src/features/auth/hooks/useAuth.js
import { useContext, useEffect } from "react";
import { AuthContext } from "../auth.context"; // Manages global auth data
import { login, register, logout, getMe } from "../services/auth.api"; // Our messengers!

export const useAuth = () => {
  const context = useContext(AuthContext);
  const { user, setUser, loading, setLoading } = context;

  const handleLogin = async ({ email, password }) => {
    setLoading(true); // Show loading spinner
    try {
      const data = await login({ email, password }); // Call the messenger
      setUser(data.user); // Save the logged-in user data
    } catch (err) {
      console.error("Login failed:", err); // Handle errors
    } finally {
      setLoading(false); // Hide loading spinner
    }
  };

  const handleRegister = async ({ username, email, password }) => {
    setLoading(true);
    try {
      const data = await register({ username, email, password }); // Call the messenger
      setUser(data.user); // Save the new user data
    } catch (err) {
      console.error("Registration failed:", err);
    } finally {
      setLoading(false);
    }
  };

  const handleLogout = async () => {
    setLoading(true);
    try {
      await logout(); // Call the messenger to log out
      setUser(null); // Clear the user data
    } catch (err) {
      console.error("Logout failed:", err);
```

```
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    const getAndSetUser = async () => {
      try {
        const data = await getMe(); // Ask the backend for current user (if logged in)
        setUser(data.user); // If found, set user
      } catch (err) {
        // Not logged in, or session expired, user will remain null
      } finally {
        setLoading(false); // Loading is complete
      }
    };
    getAndSetUser(); // Run this check once when the app loads
  }, []);

  return { user, loading, handleRegister, handleLogin, handleLogout };
};
```

Explanation:
The handleLogin, handleRegister, handleLogout functions inside useAuth are what your components call. Each function sets loading to true, calls the corresponding function from auth.api.js (our messenger service), updates the user state based on the response, and then sets loading back to false. The useEffect block calls getMe initially to see if the user is already authenticated.

2. The Auth API Service (Frontend/src/features/auth/services/auth.api.js)

This file is our dedicated messenger for authentication. It knows how to talk to the backend using axios.

```
// Frontend/src/features/auth/services/auth.api.js
import axios from "axios";

// Create a special 'axios' setup to talk to our backend
const api = axios.create({
  baseURL: "http://localhost:3000", // This is where our backend server is running!
  withCredentials: true // IMPORTANT: This tells the browser to send cookies with requests
});                     // Cookies are how the backend remembers your login status.
```

```
export async function register({ username, email, password }) {
  const response = await api.post('/api/auth/register', {
    username, email, password
  });
  return response.data; // Return the user data we got back
}

export async function login({ email, password }) {
  const response = await api.post("/api/auth/login", {
    email, password
  });
  return response.data; // Return the user data we got back
}

export async function logout() {
  const response = await api.get("/api/auth/logout");
  return response.data; // Returns a confirmation that you're logged out
}

export async function getMe() {
  const response = await api.get("/api/auth/get-me");
  return response.data; // Returns logged-in user data or an error if not logged in
}
```

Explanation:
Functions like register, login, logout, and getMe are the actual senders. They use api.post or api.get (functions from axios) to send requests to specific addresses on the backend (/api/auth/register, /api/auth/login, etc.).

The withCredentials: true setting for axios is very important here. It tells your browser to automatically include any security "cookies" that the backend might have given you during a previous login. These cookies are how the backend remembers you are logged in across different pages or even if you close and reopen your browser! The backend uses these cookies to know *who you are* when you request your reports or other private information. We will delve much deeper into how the backend handles this in Chapter 4: User Authentication System.

Conclusion

In this chapter, we've explored "Frontend Authentication," the security desk of our application. We learned how "hooks" (specifically useAuth) provide an easy way for our frontend components to manage user accounts, and how "services" (like auth.api.js) act as messengers to communicate securely with the backend. This system ensures that our app knows who you are, keeps your data private, and makes the login/logout process smooth and user-friendly.

This is just one side of the coin! While the frontend handles the visible forms and status, the real security magic – verifying passwords, creating secure sessions, and protecting user data – happens on the backend. How does the backend actually verify your identity and manage those cookies? That's what we'll uncover in our next chapter!

Next Chapter: User Authentication System

Generated by AI Codebase Knowledge Builder